
Clasificación multi etiqueta con el paquete neuralnet

El paquete `neuralnet` quizás no sea la mejor opción en R para usar redes neuronales. Si preguntas por qué, para empezar `no reconoce la fórmula típica  $y \sim .$ , no admite factores, no proporciona muchos modelos más que un MLP estándar y tiene grandes competidores` en el paquete `nnet` que parece estar mejor integrado en R y puede usarse con el paquete `caret`, y en el paquete `MXnet` que es una biblioteca de aprendizaje profundo de alto nivel que proporciona una amplia variedad de redes neuronales.

Pero aún así, creo que hay algo de valor en la facilidad de uso del paquete `neuralnet`, especialmente para un principiante, por lo tanto, lo usaremos.

Vamos a usar tanto `neuralnet` y, curiosamente, el paquete `nnet`. Vamos a cargarlos:

```
> install.packages("neuralnet")
> install.packages("nnet")
> install.packages("ggplot2")
> library(neuralnet)
> library(nnet)
> library(ggplot2)
> set.seed(10)
```

El conjunto de datos

Buscamos en UCI Machine Learning Repository y encontramos el conjunto de datos de vino.

Este conjunto de datos contiene los resultados de un análisis químico de 3 tipos diferentes de vinos. La `variable objetivo es la etiqueta del vino` que es un factor con 3 niveles (no ordenados). Los predictores son todos continuos y representan 13 variables obtenidas como resultado de mediciones químicas.

Este conjunto de datos no parece tan difícil de descifrar, incluso la descripción dice que no es tan desafiante, pero creo que está bien para practicar la clasificación. Además, es un poco menos aburrido que el conjunto de datos del iris.

Cargando los datos

Simplemente descarga el conjunto de datos desde el enlace del UCI y guárdalo como un archivo `.csv` en tu directorio de trabajo. Luego puedes cargarlo en R de la siguiente manera:

```

> # Load data and set variables names
> wines <- read.csv("wines.csv")
> names(wines) <- c("label",
+ "Alcohol",
+ "Malic_acid",
+ "Ash",
+ "Alcalinity_of_ash",
+ "Magnesium",
+ "Total_phenols",
+ "Flavanoids",
+ "Nonflavanoid_phenols",
+ "Proanthocyanins",
+ "Color_intensity",
+ "Hue",
+ "OD280_OD315_of_diluted_wines",
+ "Proline")
> head(wines)

```

	label	Alcohol	Malic_acid	Ash	Alcalinity_of_ash	Magnesium	Total_phenols	Flavanoids	Nonflavanoid_phenols
1	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28
2	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26
3	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30
4	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24
5	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39
6	1	14.20	1.76	2.45	15.2	112	3.27	3.39	0.34

	Proanthocyanins	Color_intensity	Hue	OD280_OD315_of_diluted_wines	Proline
1	2.29	5.64	1.04	3.92	1065
2	1.28	4.38	1.05	3.40	1050
3	2.81	5.68	1.03	3.17	1185
4	2.18	7.80	0.86	3.45	1480
5	1.82	4.32	1.04	2.93	735
6	1.97	6.75	1.05	2.85	1450

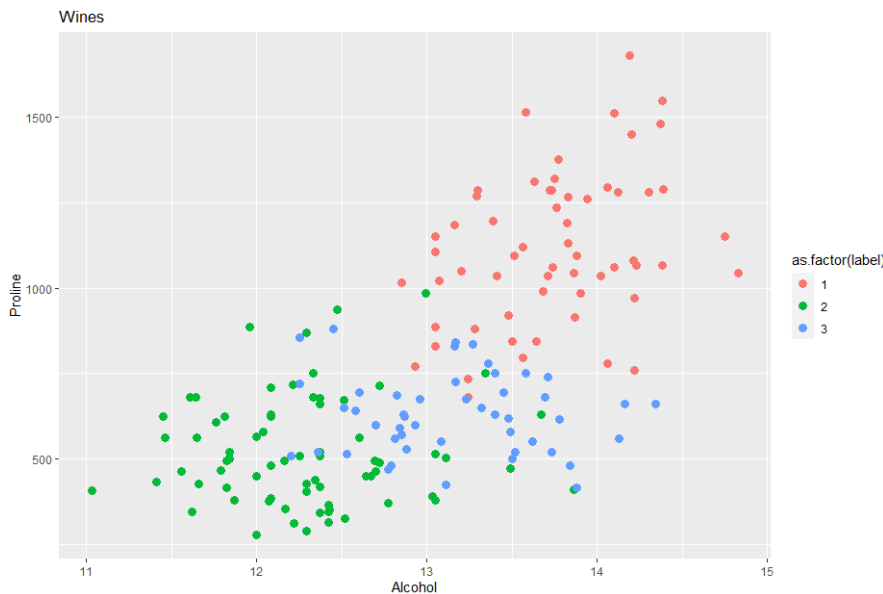
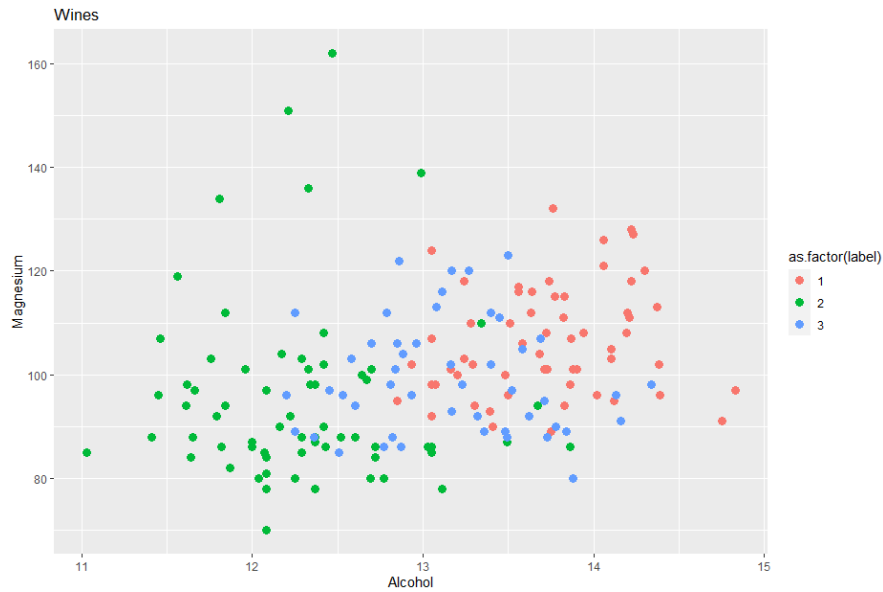
Echemos un vistazo a un gráfico de algunas de las variables:

```

> plt1 <- ggplot(wines, aes(x = Alcohol, y = Magnesium, colour = as.factor(label))) +
+ geom_point(size=3) +
+ ggtitle("Wines")
> plt2 <- ggplot(wines, aes(x = Alcohol, y = Proline, colour = as.factor(label))) +
+ geom_point(size=3) +
+ ggtitle("Wines")
> plt1
> plt2

```

Aunque los primeros dos componentes suman alrededor del 50% de la varianza total, parece que podría ser interesante ajustar algún clasificador usando solo esos dos componentes. Pero no nos distraigamos con estas hermosas tramas, pasemos al siguiente paso.



Preprocesamiento

Durante la fase de preprocesamiento, tenemos que hacer al menos las siguientes dos cosas:

Codificar las variables categóricas.

Estandarizar los predictores.

En primer lugar, codifiquemos nuestra variable de destino. La codificación de las variables categóricas es necesaria cuando se usa neuralnet ya que no le gustan los factores en absoluto. Te gritará si intentas introducir un factor (aunque me han dicho que a nnet le gustan los factores).

En el conjunto de datos del vino, la etiqueta variable contiene tres etiquetas diferentes: 1, 2 y 3.

La práctica habitual, hasta donde yo sé, es **codificar las variables categóricas como un vector "one hot"**. Por ejemplo, **si tuvieras tres clases, como en este caso, necesitarías reemplazar la variable de etiqueta con tres variables.**

En este caso, la primera observación se etiquetaría como 1, la segunda se etiquetaría como 2, y así sucesivamente. Irónicamente, **el paquete nnet proporciona una función para realizar esta codificación de forma sencilla:**

```
> # Encode as a one hot vector multilabel data
> train <- cbind(wines[, 2:14], class.ind(as.factor(wines$label)))
> # Set labels name
> names(train) <- c(names(wines)[2:14], "l1", "l2", "l3")
```

Por cierto, **dado que todos los predictores son continuos, no necesitas codificar ninguno de ellos**, sin embargo, en caso de que lo necesites, puedes aplicar la misma estrategia aplicada anteriormente a todos los predictores categóricos. A menos, por supuesto, que desees probar algún otro tipo de codificación personalizada.

Ahora **estandaricemos los predictores** en el intervalo $[0-1]$ aprovechando la función `lapply`:

```
> # Scale data
> scl <- function(x){ (x - min(x))/(max(x) - min(x)) }
> train[, 1:13] <- data.frame(lapply(train[, 1:13], scl))
> head(train)
```

	Alcohol	Malic_acid	Ash	Alcalinity_of_ash	Magnesium	Total_phenols	Flavanoids	Nonflavanoid_phenols
1	0.8421053	0.1916996	0.5721925	0.25773196	0.6195652	0.6275862	0.5738397	0.2830189
2	0.5710526	0.2055336	0.4171123	0.03092784	0.3260870	0.5758621	0.5105485	0.2452830
3	0.5605263	0.3201581	0.7005348	0.41237113	0.3369565	0.6275862	0.6118143	0.3207547
4	0.8789474	0.2391304	0.6096257	0.31958763	0.4673913	0.9896552	0.6645570	0.2075472
5	0.5815789	0.3656126	0.8074866	0.53608247	0.5217391	0.6275862	0.4957806	0.4905660
6	0.8342105	0.2015810	0.5828877	0.23711340	0.4565217	0.7896552	0.6434599	0.3962264

	Proanthocyanins	Color_intensity	Hue	OD280_OD315_of_diluted_wines	Proline	l1	l2	l3
1	0.5930599	0.3720137	0.4552846	0.9706960	0.5613409	1	0	0
2	0.2744479	0.2645051	0.4634146	0.7802198	0.5506419	1	0	0
3	0.7570978	0.3754266	0.4471545	0.6959707	0.6469330	1	0	0
4	0.5583596	0.5563140	0.3089431	0.7985348	0.8573466	1	0	0
5	0.4447950	0.2593857	0.4552846	0.6080586	0.3259629	1	0	0
6	0.4921136	0.4667235	0.4634146	0.5787546	0.8359486	1	0	0

Ajuste del modelo con neuralnet

Ahora es el momento de ajustar el modelo.

Como recordarás, a neuralnet no le gusta la fórmula $y \sim ..$. No temas, puedes construir la fórmula para usarla en un simple paso:

```
> # Set up formula
> n <- names(train)
> f <- as.formula(paste("l1 + l2 + l3 ~", paste(n[!n %in% c("l1","l2","l3")], collapse = " + ")))
> f
l1 + l2 + l3 ~ Alcohol + Malic_acid + Ash + Alcalinity_of_ash +
  Magnesium + Total_phenols + Flavanoids + Nonflavanoid_phenols +
  Proanthocyanins + Color_intensity + Hue + OD280_OD315_of_diluted_wines +
  Proline
```

Ten en cuenta que los caracteres del vector no se pegan a la derecha del símbolo "~".

Solo recuerda verificar que la fórmula sea correcta y luego estarás listo para comenzar.

Entrenemos la red neuronal con el conjunto de datos completo. Debería tomar muy poco tiempo para converger. Sin embargo, si no estandarizaste los predictores, podría tomar mucho más.

```
> nn <- neuralnet(f,
+ data = train,
+ hidden = c(13, 10, 3),
+ act.fct = "logistic",
+ linear.output = FALSE,
+ lifesign = "minimal")
hidden: 13, 10, 3  thresh: 0.01  rep: 1/1  steps: 89 error: 0.02455 time: 0.21 secs
```

Ten en cuenta que establecimos el argumento linear.output en FALSE para decirle al modelo que quiero aplicar la función de activación act.fct y que no estamos haciendo una tarea de regresión. Luego configuramos la función de activación en logística (que, por cierto, es la opción predeterminada) para aplicar la función logística. La otra opción disponible es Tanh, pero el modelo parece funcionar un poco peor, así que optamos por la opción predeterminada. Hasta donde sabemos, estas dos son las dos únicas opciones disponibles, no hay una función "relu" disponible, aunque parece ser una función de activación común en otros paquetes.

En cuanto a la cantidad de neuronas ocultas, probamos alguna combinación y la utilizada parece funcionar un poco mejor que las otras (alrededor del 1% de diferencia de precisión en el puntaje de validación cruzada).

Al usar el método de trazado incorporado, puedes obtener una visión visual de lo que realmente está sucediendo dentro del modelo, sin embargo, creo que el trazado no es tan útil.

```
> plot(nn)
```

Echemos un vistazo a la precisión en el conjunto de entrenamiento:

```
> # Compute predictions
> pr.nn <- compute(nn, train[, 1:13])
> # Extract results
> pr.nn_ <- pr.nn$net.result
> head(pr.nn_)
      [,1]      [,2]      [,3]
[1,] 0.9919754 0.003020934 5.042477e-06
[2,] 0.9913000 0.003559137 4.524784e-06
[3,] 0.9920582 0.003003758 4.938348e-06
[4,] 0.9923104 0.002871484 5.158712e-06
[5,] 0.9872282 0.005115707 5.583214e-06
[6,] 0.9923760 0.002874163 4.903709e-06
>
> # Accuracy (training set)
> original_values <- max.col(train[, 14:16])
> pr.nn_2 <- max.col(pr.nn_)
> mean(pr.nn_2 == original_values)
[1] 1
```

100% no está mal! Pero espera, esto puede deberse a que nuestro modelo sobre ajustó los datos, además evaluar la precisión en el conjunto de entrenamiento es una especie de trampa ya que el modelo ya “sabe” (o debería saber) las respuestas. Para evaluar la “verdadera precisión” del modelo, debes realizar algún tipo de validación cruzada.

Validación cruzada del clasificador

Hagamos una validación cruzada del modelo utilizando la validación cruzada de **10 fold cross** con la siguiente división de entrenamiento y prueba: el 95 % del conjunto de datos se usará como conjunto de entrenamiento, mientras que el 5 % restante se usará como conjunto de prueba.

Solo por curiosidad, decidimos ejecutar una ronda **LOOCV** también. En caso de que desees ejecutar esta técnica de validación cruzada, simplemente configura la variable de proporción en 0.995: esto seleccionará solo una observación como conjunto de prueba y dejará todas las demás observaciones como conjunto de entrenamiento. Al ejecutar LOOCV, deberías obtener resultados similares a los de la validación cruzada de 10 veces.

```

> # Crossvalidate
> set.seed(10)
> k <- 10
> outs <- NULL
> proportion <- 0.995
> library(plyr)
> pbar <- create_progress_bar('text')
> pbar$init(k)
> for(i in 1:k)
+ {
+   index <- sample(1:nrow(train), round(proportion*nrow(train)))
+   train_cv <- train[index, ]
+   test_cv <- train[-index, ]
+   nn_cv <- neuralnet(f,
+     data = train_cv,
+     hidden = c(13, 10, 3),
+     act.fct = "logistic",
+     linear.output = FALSE)
+   # Compute predictions
+   pr.nn <- compute(nn_cv, test_cv[, 1:13])
+   # Extract results
+   pr.nn_ <- pr.nn$net.result
+   # Accuracy (test set)
+   original_values <- max.col(test_cv[, 14:16])
+   pr.nn_2 <- max.col(pr.nn_)
+   outs[i] <- mean(pr.nn_2 == original_values)
+   pbar$step()
+ }
> mean(outs)
[1] 1

```

100%, impresionante! La próxima vez que te inviten a una velada relajante que incluya una competencia de cata de vinos, ¡creo que definitivamente deberías traer tu computadora portátil como concursante!

De hecho, este conjunto de datos no es el más desafiante, creo que con algunos ajustes más se podría lograr una mejor puntuación de validación cruzada. No obstante, espero que hayas encontrado útil esta práctica.